

pulse-scan: Low-Level Design v1

1. Context

What this is

A read-only scanner for RAG knowledge bases. Connects to a user's existing vector store, analyzes the corpus, and produces a health report identifying stale chunks, near-duplicates, contradictions between chunks, and supersession candidates. Outputs both machine-readable (JSON/Parquet) and human-readable (Streamlit dashboard) artifacts.

What this is not

Not a vector store. Not a write-time gateway. Not a query-time reranker. Not a generic observability tool for LLM traces. Future versions may add gateway/shadow modes that share this scanner's control plane, but they are out of scope for v1.

Who it's for

Engineers maintaining production RAG systems with anywhere from 10k to 10M chunks. They have a vector store (Chroma in v1; others in future versions), an embedding model whose output is in that store, and they suspect their corpus is rotting but have no way to see it.

The problem being solved

Vector stores own retrieval. Eval tools own output quality. No tool owns corpus integrity. Stale chunks, contradictions between chunks, and abandoned content silently degrade RAG output and are invisible to existing tooling. pulse-scan is the integrity layer.

The defensible novelty

The math is borrowed (Grofsky 2025 for time-decay, standard NLI for contradiction, HDBSCAN for clustering, conformal prediction for calibration). The form factor is novel: a vector-store-agnostic scanner that produces a corpus health report and dashboard. The visual language for "what does corpus health look like" is unclaimed and is treated as a first-class deliverable.

Operating model

Runs on the user's infrastructure. Distributed as a Docker image with a CLI and an embedded Streamlit dashboard. One-time scans for first-time use, scheduled scans (daily/weekly) for ongoing monitoring. We run no compute on behalf of users. Cost is bounded and predictable.

Hard requirements

- **GPU is required.** No CPU fallback. If `(torch.cuda.is_available())` is false at scan start, the scanner exits with an error pointing to GPU setup docs.
- **PII handling is out of scope for v1.** The dashboard shows chunk text plainly. README must warn users not to run on data they wouldn't be comfortable seeing on their own

2. Architecture

Three planes

Data plane (external, read-only). The user's vector store. Accessed through adapters. Never written to.

Control plane (ours). Local persistent state: scan history, chunk metadata cache, embeddings cache, cluster assignments, contradiction edges, dedup groups, calibration state, scoring history. DuckDB (single file, portable, fast for analytical queries) plus a memory-mapped numpy file for embeddings.

Compute plane (ours, ephemeral). The scan pipeline itself. NLI inference, clustering, scoring. Runs during a scan, idle between. GPU required.

Component diagram



Outputs

JSON report (machine-readable)

Streamlit dashboard (human-readable, local)

3. Stages

Stage 0: Ingestion + cache

Purpose. Connect to the user's vector store, fetch chunks, populate or refresh the local cache.

Inputs. Vector store credentials and connection config. A `pulse.config.yaml` declaring which collections to scan, which metadata field holds timestamps, optional retrieval log path.

Outputs. Local cache: chunk records in DuckDB, embeddings in a memory-mapped numpy file. Each chunk has a content hash for change detection.

Algorithm.

1. Connect via the store-specific adapter.
2. Fetch chunks in batches (adapter-defined batch size).
3. For each chunk, compute `xxhash(text)` as content key.
4. Look up chunk by `(store_id, chunk_id)` in DuckDB. Three cases:
 - Not seen before: insert, store embedding at next memmap offset, set `first_seen_by_pulse = now()`.
 - Seen, hash unchanged: skip (no update).
 - Seen, hash changed: update text and embedding, increment version, record change in scan history.
5. Mark chunks present in DuckDB but absent from this fetch as `deleted_at = now()`.

Embedding-model identification.

Resolution order:

1. Explicit `embedding_model:` in config.
2. Metadata field if present (`embedding_model`, `model`, `embedder` — common patterns).
3. Dimension-based guess from a built-in known-models table:

```
python
```

```
KNOWN_MODELS = {
    1536: ['openai-ada-002', 'openai-text-embedding-3-small'],
    3072: ['openai-text-embedding-3-large'],
    768: ['sentence-transformers/all-mpnet-base-v2', 'BGE-base', 'E5-base'],
    1024: ['BGE-large', 'E5-large', 'Voyage-2', 'Cohere-embed-v3'],
    384: ['sentence-transformers/all-MiniLM-L6-v2'],
}
```

If dimension matches multiple known models, log them all and use the most conservative threshold default. **Failure mode documented:** if pulse picks the wrong model, dedup may be over- or under-aggressive. Override via `embedding_model:` in config.

Timestamp resolution. Real corpora rarely have clean `created_at`. The adapter resolves a timestamp per chunk in this order:

1. The user-declared `timestamp_field` from config, if present and parseable.
2. UUIDv1/ULID-embedded timestamp if the chunk ID is one of those formats.
3. `first_seen_by_pulse` as fallback.

The `timestamp_source` is recorded per chunk so downstream stages know which signal they're working with.

Optimizations.

- xxhash for content addressing (faster than SHA, sufficient for non-cryptographic dedup).
- Memory-mapped numpy array for embeddings: zero RAM cost for unused chunks, fast random access.
- Incremental: subsequent scans only process changed/new chunks.

Gotchas to handle.

- Embedding dimensions vary by model. Detect on first scan, fail loudly if dimensions change between scans (indicates user changed embedding models, needs explicit migration).
- Pinecone returns lists; convert to numpy in batched bulk (single-chunk conversion is a real bottleneck at scale).
- Some stores don't expose embeddings at fetch time. Adapter must handle both: fetch with embeddings, or fetch IDs then fetch embeddings separately.

Bounds. Time-bounded by config (default 30 minute fetch budget). On timeout, save partial state and resume next scan.

Stage 0.5: Calibration

Purpose. Embedding similarity distributions are model-dependent. A cosine of 0.85 means "near duplicate" for one model and "vaguely related" for another. Hardcoded thresholds break across users. Calibration computes per-corpus thresholds before any thresholded operation runs.

When it runs. First scan always. Subsequent scans only if (a) embedding dimensions changed, (b) corpus size grew by more than 50% since last calibration, or (c) user manually triggers recalibration.

Inputs. A sample of chunk pairs from the cache.

Outputs. A `calibration` table row in DuckDB with: `dedup_cosine_threshold`, `contradiction_candidate_threshold`, `cluster_min_density`, plus the sample distributions used to derive them.

Algorithm.

1. Sample N random chunks ($N = \min(2000, \text{corpus_size})$). Compute pairwise cosine distribution. This is the **baseline distribution** — typical cosine between random pairs.
2. For each sampled chunk, find its top-10 nearest neighbors via a temporary HNSW index (hnsplib). Compute cosine to each. This is the **neighbor distribution** — what "similar" looks like.
3. Set thresholds from the distributions:
 - `dedup_cosine_threshold`: 99.5th percentile of the neighbor distribution.
 - `contradiction_candidate_threshold`: 90th percentile of the neighbor distribution. Sets the cosine floor below which we don't bother running NLI.
 - `cluster_min_density`: median of neighbor distribution, used as input to HDBSCAN min cluster size heuristic.
4. Persist with the calibration sample's hash so we can detect when re-calibration is needed.

Why this matters. Without it, the dedup stage either produces zero hits (threshold too high) or floods (threshold too low) on different embedding models. Calibration is what makes the tool work across users.

Gotcha. With very small corpora (<500 chunks), the sample is the whole corpus and distributions are noisy. Fall back to model-specific defaults from the known-models table.

Bounds. $O(N)$ with $N=2000$. Runs in seconds.

Note on calibration vs confidence. This stage calibrates the *cosine thresholds* used for candidate generation. The *NLI confidence* calibration is a separate, dashboard-driven loop described in Stage 4.

Stage 1: Deduplication

Purpose. Find near-duplicate chunks. Critical because duplicates poison contradiction detection (NLI sees paraphrases as contradictions, killing precision).

Two-channel detection. Text-level and embedding-level, both required because each catches different cases.

Channel A: Text dedup (MinHash + LSH).

1. For each chunk, generate word 3-grams.
2. Compute MinHash signature (datasketch, 128 permutations).
3. LSH bucketing with threshold 0.5 (Jaccard).
4. Within each bucket, all pairs are candidates.
5. Verify candidates by re-computing exact Jaccard on shingle sets. Keep pairs with Jaccard > 0.7.

Channel B: Embedding dedup (HNSW + cosine).

1. Build an in-memory HNSW index over all embeddings (hnswlib, ~30 seconds for 1M chunks).
2. Query each chunk against the index, k=10.
3. Candidates: pairs with cosine > `dedup_cosine_threshold` (from calibration).
4. Exclude self-matches.

Merge. Union the candidates from both channels. A pair flagged by either channel is a near-duplicate.

Group resolution. Connect candidate pairs into duplicate groups via union-find. Each group gets a "canonical" chunk: the newest by resolved timestamp, with content-length tiebreaker.

Outputs. `dedup_groups` table: `(group_id, canonical_chunk_id, member_chunk_ids, detection_channels)`.

Why two channels. Text channel catches lexical duplicates (same content, formatting differences). Embedding channel catches semantic duplicates (paraphrases, translations, summary-vs-original). Either alone misses cases.

Gotchas.

- The "canonical" chunk choice affects downstream contradiction detection. If canonical is wrong (older chunk picked over newer), supersession is reversed. Log the choice and let users override in dashboard.
- Translation pairs (English chunk + Spanish chunk of same content) are dups by embedding but not by text. This is correct behavior, but users should be able to mark languages and decide policy.

Bounds. $O(N \log N)$ via LSH and HNSW. At 1M chunks, ~5 minutes total.

Stage 2: Clustering

Purpose. Restrict pairwise NLI to within-cluster pairs. Without this, NLI is $O(N^2)$ and infeasible.

Algorithm.

1. **Dimensionality reduction.** UMAP from embedding_dim $\rightarrow$ 50d. Starting params: `(min_dist=0.1, n_neighbors=15, random_state)` set for reproducibility.
2. **Clustering.** HDBSCAN on the 50d projection. `(min_cluster_size = max(5, sqrt(N)/4)` as starting heuristic, `(min_samples=3)`.
3. **Cache the UMAP model** to disk. Re-fit only when corpus has shifted significantly (>20% new chunks since last fit).
4. Output cluster assignment per chunk; chunks labeled -1 are "noise" — mark them as `(cluster_id = NULL)` and treat as orphan candidates downstream.

Why UMAP first. Clustering directly in 768d/1536d space is slow and the curse of dimensionality hurts cluster quality. UMAP preserves local structure (which is what we need: "is this chunk near these other chunks") much better than PCA at low target dims.

Why HDBSCAN over k-means. No need to specify k. Handles variable cluster density. Produces noise label, which becomes our orphan signal.

Gotchas.

- UMAP is stochastic. Caching the fitted model is non-negotiable for stable cluster assignments across scans.
- `(min_cluster_size)` materially affects results. Provide an `(auto_tune_clustering)` flag that tries 3 values and picks the one with best silhouette score, gated behind explicit user opt-in (it's expensive).
- Some chunks legitimately belong to two topics. v1 accepts hard assignments; v2 may add soft clustering. Note in the report when a chunk's distance to its second-nearest cluster centroid is close to its primary.

GPU optimization. cuML's UMAP is 10-50x faster on GPU at >100k chunks. Provide a GPU path; default to CPU UMAP for portability of the clustering stage (NLI still requires GPU separately).

Outputs. `(clusters)` table: `(cluster_id, chunk_id)`; `(cluster_centroids)` table for centroid storage.

Bounds. UMAP + HDBSCAN: ~2 min on CPU at 100k chunks, ~30 sec on GPU at 1M chunks.

Stage 3: Triage

Purpose. Decide which chunks get expensive deep analysis (NLI) this scan. Pure ranking problem.

Data structure. A min-heap (Python `heapq`) keyed by negative priority score. Persisted between scans so chunks deferred this run get higher priority next run.

Scoring function (v1, weighted sum).

```
priority(c) =  
    w1 * age_factor(c)  
  + w2 * cluster_growth(c.cluster, since=last_scan)  
  + w3 * cosine_drift(c, c.cluster.recent_centroid)  
  + w4 * retrieval_recency_factor(c)           # if logs available  
  + w5 * time_since_last_pulse_scan(c)  
  + w6 * never_seen_in_nli_pass(c)           # boost first-time chunks
```

Initial weights from heuristic; configurable. Track which signals predict actual contradictions in scan history; this becomes training data for a learned ranker in v0.3+.

Cost-bounded scan budget. Not "scan top 1000 chunks." Instead, "scan until cost budget exhausted." Each operation has a cost:

- NLI pair: 1 cost unit
- Per-chunk neighbor lookup: 0.05 cost units
- Cluster centroid recompute: 0.5 units per cluster

Default budget configurable, suggested 50k cost units per scan. Predictable scan time across corpora of different shapes.

Outputs. `trriage_log` table: ordered list of chunks selected for this scan, with priority components recorded for later analysis.

Gotcha. Weighted sums are arbitrary. Log every score component every scan. The instrumentation now is what enables the learned ranker later. **Do not skip the logging.**

Stage 4: Contradiction detection

Purpose. Identify chunk pairs whose claims contradict each other.

Three parallel detectors.

Detector A: NLI (primary).

1. For each chunk in the scan queue, find candidate partners: same cluster, top-k by cosine (k from config, default 5), excluding pairs from the same dedup group, with cosine >

`contradiction_candidate_threshold` from Stage 0.5.

2. Generate ordered pairs (NLI is asymmetric; check both directions).
3. Batch into groups of 64 (or configurable based on GPU memory).
4. Run inference: DeBERTa-v3-base fine-tuned on MNLI from HuggingFace.
5. Output: `(entailment, neutral, contradiction)` probabilities per ordered pair.
6. A pair is a "contradiction candidate" if either direction has contradiction probability above threshold (see calibration loop below).

Detector B: Numeric contradiction (regex prefilter). NLI models are weak on numbers. Run a parallel detector:

1. Extract numeric facts from each chunk: amount + unit + entity. Patterns like `$X`, `X%`, `X` days, `X cents`, `version X`.
2. For each pair already passing the cosine candidate filter, check if both have a numeric fact about the same entity-type but different values.
3. Flag as numeric contradiction with confidence based on entity-match strength.

Detector C: Versioned-identifier contradiction. Specific case but high-value: SDK versions, API versions, package names. Pattern-match `@scope/package`, `package-vN`, `package@x.y.z`. If two chunks reference the same package with different versions in similar context, flag as supersession candidate.

Confidence calibration loop (dashboard-driven).

This is the v1 approach: no synthetic seed set, no labels shipped with the tool. Calibration emerges from user feedback in the dashboard.

Cold start (no labels yet):

- All NLI hits with raw contradiction probability > 0.5 are surfaced.
- Dashboard displays them with an "uncalibrated" banner.
- `calibration_state = 'uncalibrated'` on each contradiction record.
- `calibrated_confidence = NULL`. Dashboard reads `raw_score` for display.

During use:

- Every contradiction in the dashboard has approve / reject / skip buttons.
- Clicks write to `contradictions.user_resolution` (`confirmed | rejected | skipped`).

Progressive calibration:

- Once 30 globally-labeled examples exist (or 15 per cluster), conformal calibration kicks in (MAPIE library).

- Past contradictions get `calibration_state = 'calibrated'` and a populated `calibrated_confidence`.
- New contradictions get calibrated immediately.
- **Raw scores are never overwritten.** Both `raw_score` and `calibrated_confidence` are stored; the dashboard chooses which to display based on `calibration_state`.

Why this is better than a shipped seed set.

- No seed set to maintain.
- No onboarding flow asking users to label things before they see value.
- Users see hits immediately, natural review behavior generates labels.
- Calibration improves continuously.

The dashboard banner during cold start frames it positively: *"This scan ran without calibration. Review the findings to teach pulse what counts as a real contradiction in your corpus."*

Why three detectors. Each catches what the others miss. NLI handles semantic contradictions but misses numbers. Numeric detector catches "2.4% vs 2.9%". Version detector catches the SDK supersession case. Union is much better than any single detector.

Gotchas.

- Numeric extraction is fragile. Currency symbols vary. "\$5" and "5 dollars" should match. Use a small library (`price-parser` for money, custom for percentages). Plan for ongoing improvement.
- NLI is asymmetric. Cosine prefilter (>0.7) prevents orthogonal pairs from being noise.

GPU is required here. At 1M chunks with avg 5 candidate pairs per scanned chunk, NLI runs into millions of inferences. CPU is hours; GPU is minutes. No CPU fallback.

Outputs. `contradictions` table: `chunk_a_id, chunk_b_id, detector, raw_score, calibrated_confidence, calibration_state, direction, scan_run_id, user_resolution, resolved_at`.

Stage 5: Staleness scoring

Purpose. Score every chunk on a continuous staleness scale, separately from contradiction.

Algorithm. Per chunk:

```

staleness(c) = sigmoid(
    a1 * age_decay(c)                # Grofsky half-life
  + a2 * cluster_drift(c)            # cosine to recent centroid
  + a3 * contradiction_evidence(c)  # any high-confidence
contradictions
  + a4 * retrieval_abandonment(c)    # never retrieved or
last_retrieved old
  + a5 * supersession_evidence(c)    # newer chunk same dedup-cluster
)

```

Output: continuous score in [0,1], plus a label (`fresh` < 0.3, `aging` 0.3-0.6, `stale` 0.6-0.85, `abandoned` > 0.85).

Per-collection half-lives. Different content types decay at different rates. v1 accepts user-configured half-lives per collection. v2 will learn them from contradiction-arrival rate per collection. Stub the learning hook in v1.

Gotcha — time-bounded content. Chunks like "spring promotion" are stale by content, not by corpus dynamics. Detecting this requires LLM-based temporal scoping. **Out of scope for v1.** Note in the report as "low confidence stale, possibly time-bounded — manual review."

Outputs. `chunks` table updated with: `staleness_score`, `staleness_label`, `staleness_components` (per-component breakdown stored as JSON for explainability).

Stage 6: Output and dashboard

Two outputs.

Machine-readable: scan report.

- Format: JSON (default) or Parquet (for large corpora).
- Schema: scan metadata, chunk-level records, contradiction edges, dedup groups, cluster summaries.
- Stable schema with version field — downstream tools depend on it.

Human-readable: Streamlit dashboard.

Streamlit chosen for speed of build and simplicity. Reads directly from the DuckDB control plane. Runs locally on the user's laptop, not on AWS.

Required views in v1:

- **Corpus map:** 2D UMAP scatter plot, points colored by staleness score, sized by retrieval count if available. Click a point to drill in. Click-and-drag a region to filter. Use plotly or pydeck within Streamlit.

- **Contradiction graph:** Force-directed graph of contradiction edges. Edge thickness = confidence (or raw score during cold start). Hover for chunk text. Filter by detector type. Each edge has approve / reject / skip buttons feeding the calibration loop.
- **Findings table:** Sortable, filterable list of chunks. Default sort: staleness desc. Columns: chunk preview, cluster, staleness label, contradictions count, last retrieved, age.
- **Chunk drill-down:** Click any chunk → full text, neighbors with cosine, contradicting chunks, supersession candidates, full text of related chunks, timeline of when this chunk was first/last seen.
- **Scan summary:** counts by category, time-series of corpus health across scans (once 2+ scans exist). Calibration state banner during cold start.

Visualization is treated as USP. "What does corpus health look like" is unclaimed territory. Invest in this; do not punt to "we'll add a dashboard later."

Gotchas.

- Scatter at scale (>100k chunks) is unusable without drill-down. Implement region-zoom from day one.
- v1 dashboard shows chunk text plainly. README must warn about PII. Redaction lands in v1.1.

4. Adapters

v1 ships with one production adapter and one fixture adapter:

- **Chroma adapter** (production target for v1).
- **Local fixture adapter** for testing — reads chunks from a directory of JSON files. Needed for the synthetic benchmark and for users who want to evaluate pulse before connecting it to anything.

Adapter interface (Python protocol):

```
python
```

```
class VectorStoreAdapter(Protocol):
    def list_collections(self) -> list[CollectionInfo]: ...
    def fetch_chunks(
        self,
        collection: str,
        batch_size: int,
        since: Optional[datetime] = None,
    ) -> Iterator[ChunkBatch]: ...
    def get_embeddings(
        self,
        collection: str,
        chunk_ids: list[str],
    ) -> np.ndarray: ...
    @property
    def supports_embeddings_in_fetch(self) -> bool: ...
    @property
    def supports_metadata_filtering(self) -> bool: ...
```

Future adapters (Qdrant, pgvector, Pinecone, Weaviate) are post-v1.

5. Storage / data model

DuckDB tables (control plane):

```
sql
```

```

chunks (
  store_id TEXT, collection TEXT, chunk_id TEXT,
  text TEXT, content_hash TEXT,
  resolved_timestamp TIMESTAMP, timestamp_source TEXT,
  embedding_offset BIGINT,
  cluster_id INTEGER,
  staleness_score DOUBLE, staleness_label TEXT,
  staleness_components JSON,
  first_seen_by_pulse TIMESTAMP, last_seen_by_pulse TIMESTAMP,
  deleted_at TIMESTAMP,
  version INTEGER,
  PRIMARY KEY (store_id, collection, chunk_id)
)

dedup_groups (
  group_id INTEGER, canonical_chunk_id TEXT,
  member_chunk_ids JSON, detection_channels JSON
)

clusters (cluster_id INTEGER, chunk_id TEXT)
cluster_centroids (cluster_id INTEGER, centroid BLOB, n_chunks INTEGER)

contradictions (
  chunk_a TEXT, chunk_b TEXT,
  detector TEXT, -- 'nli', 'numeric', 'version'
  raw_score DOUBLE,
  calibrated_confidence DOUBLE, -- NULL during cold start
  calibration_state TEXT, -- 'uncalibrated' | 'calibrated'
  direction TEXT, -- 'a->b', 'b->a', 'both'
  scan_run_id TEXT,
  user_resolution TEXT, -- NULL | 'confirmed' | 'rejected' | 'skipped'
  resolved_at TIMESTAMP
)

scan_runs (run_id TEXT, started_at, finished_at, config JSON, stats JSON)

calibration (
  scan_run_id TEXT, sample_hash TEXT,
  dedup_threshold DOUBLE, contradiction_candidate_threshold DOUBLE,
  cluster_min_density DOUBLE,
  distributions JSON
)

trriage_log (
  scan_run_id TEXT, chunk_id TEXT, priority DOUBLE,
  components JSON,

```

```
was_scanned BOOLEAN
)
```

Embeddings. A single memory-mapped numpy file (`embeddings.f32.npy`) of shape `(N, D)`. Chunks reference rows by `embedding_offset`. Append-only with periodic compaction (separate maintenance command).

6. Configuration

A single `pulse.config.yaml`:

```
yaml
```

```
store:
  type: chroma
  connection: { ... }

# Optional: explicit embedding model override
embedding_model: openai-text-embedding-3-small

collections:
  - name: docs
    timestamp_field: created_at
    half_life_days: 90
  - name: pricing
    timestamp_field: createdAt
    half_life_days: 18

scan:
  cost_budget: 50000
  nli_batch_size: 64
  contradiction_candidates_per_chunk: 5
  enable_numeric_detector: true
  enable_version_detector: true

clustering:
  min_cluster_size: auto
  use_gpu: false

inference:
  nli_model: deberta-v3-base-mnli
  device: cuda # cuda only in v1; cpu raises error

dashboard:
  host: 127.0.0.1
  port: 8080
```

7. Hardware requirements

- **GPU required.** No CPU fallback for inference. Any NVIDIA GPU with ≥ 8 GB VRAM. T4, 3060, 3090, A10, A100 all work.
- **RAM:** 8GB minimum, 16GB+ recommended for >500k chunks.
- **Disk:** ~5KB per chunk in the control plane, plus 6KB per chunk for embeddings (assuming 1536d float32). At 1M chunks, ~11GB.
- **GPU not available?** Scanner exits at startup with clear error and link to GPU setup docs.

8. AWS deployment (day-0)

Cheap and light. v1 priority is "test working, not scaling."

Components

ECS cluster: EC2 launch type (Fargate doesn't support GPU). One task definition for the scanner.

Compute: g4dn.xlarge. Single NVIDIA T4, 16GB VRAM, 4 vCPU, 16GB RAM. ~\$0.50/hr on-demand, ~\$0.15/hr spot. Smallest GPU instance type AWS sells. For v1 corpora (10k-100k chunks), a scan completes in 15-45 minutes.

Storage:

- **EFS volume** mounted to the ECS task for the control plane (DuckDB file + embeddings memmap). Persists between scans without copying state. ~\$0.30/GB/month. For v1 (~12GB total), ~\$4/month.

Networking:

- ECS task in a private subnet with NAT for outbound calls (vector store, model weights download).
- No inbound traffic needed — batch job.
- **Dashboard runs locally on user's laptop**, not on AWS. Pulls DuckDB file from EFS via a fetch script. Hosting Streamlit on AWS adds ALB, security groups, TLS, auth — not worth it for v1.

Triggering:

- Manual `aws ecs run-task` for v1.
- EventBridge cron → ECS RunTask later, when stable.

Container registry: ECR for the Docker image.

IAM:

- ECS task role with read access to vector store (network reachability).
- Read/write to EFS volume.
- Read from Secrets Manager for API keys / vector store creds.
- Nothing else.

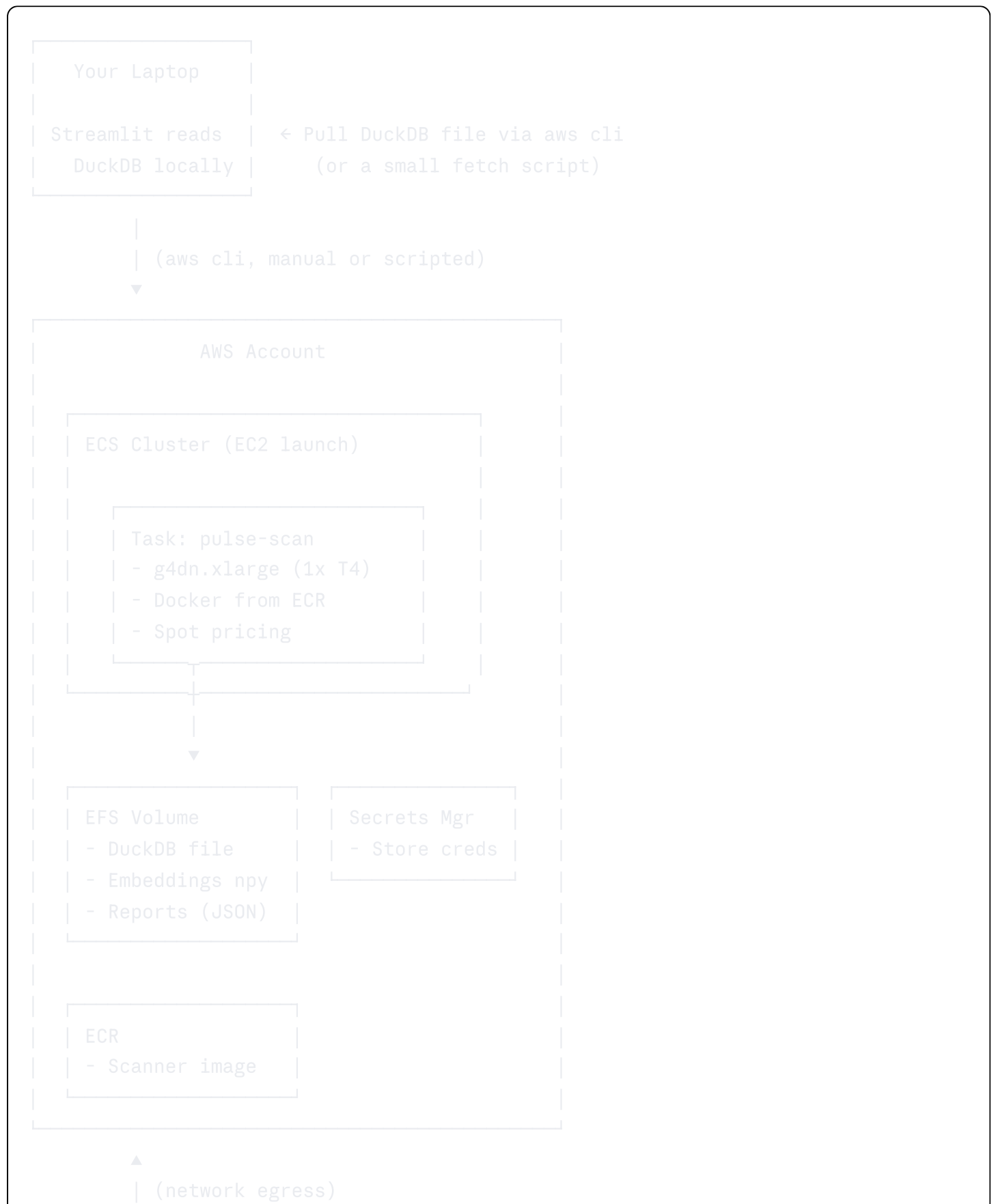
Cost estimate

Weekly scans of a 50k-chunk corpus:

- Compute: g4dn.xlarge spot, ~30 min/scan, 4 scans/month = 2 hrs = ~\$0.30
- Storage: EFS, ~12GB = ~\$4
- ECR, networking, misc: ~\$2

~\$6-7/month. Daily scans: ~\$10/month.

Architecture diagram



Your Vector
Store (Chroma)

Deliberately excluded from v1

- Scheduler (cron via EventBridge). Manual triggering is fine.
- Hosted dashboard. Local Streamlit only.
- Multi-account / multi-region. Single account, single region.
- Monitoring/alerting. CloudWatch logs from the ECS task is enough.
- CI/CD for the scanner image. Manual `docker build && docker push`.
- Terraform/CDK from day one. Shell scripts wrapping `aws cli` until the third deployment.

Hidden gotchas

- **g4dn spot availability varies by AZ.** Bid across 2-3 AZs. Specify capacity providers properly in the ECS task.
- **First scan downloads NLI model (~430MB) from HuggingFace.** Bake into the Docker image; don't pull at runtime.
- **EFS has a small ongoing cost even when unused.** If you go dormant, the bill keeps ticking. Acceptable, but know it.
- **Vector store reachability.** If Chroma is on a different VPC, peering or public access is needed. Easiest: run Chroma somewhere internet-reachable with auth.

9. v1 scope (firm)

In:

- All 7 stages (0 through 6)
- Chroma adapter + fixture adapter
- All three contradiction detectors (NLI, numeric, version)
- Dashboard-driven calibration loop (no shipped seed labels)
- Streamlit dashboard with all five required views
- CLI (`pulse scan`), (`pulse dashboard`), (`pulse calibrate`)
- Synthetic 50-chunk benchmark corpus for testing
- AWS ECS deployment as described in §8

Out (deferred):

- Other vector store adapters (Qdrant, pgvector, Pinecone, Weaviate)
 - Topic evolution detection
 - Time-bounded content detection
 - Learned priority ranker (v1 uses weighted sum + logging)
 - Per-collection half-life learning (v1 accepts user config, logs the data needed to learn)
 - Gateway/shadow modes
 - Auth, multi-user, hosted dashboard
 - PII redaction in dashboard
 - CPU inference fallback
-

10. Build order

The stages aren't built in numerical order. Build for working ends-to-ends as fast as possible, then deepen.

1. **Fixture adapter + ingestion (Stage 0) + minimal DuckDB schema.**
2. **The synthetic 50-chunk benchmark corpus.** Hand-written. Includes near-duplicates, evolving pricing, API version supersession, isolated chunks, time-bounded content.
3. **Calibration (Stage 0.5)** — minimal version, just enough to produce thresholds.
4. **Embedding-channel dedup only (Stage 1, half).**
5. **Clustering (Stage 2).**
6. **NLI contradiction detection only (Stage 4, one detector).** Skip triage; scan everything in the small benchmark.
7. **Minimal staleness scoring (Stage 5).**
8. **JSON report (Stage 6, half).**

After step 8: working end-to-end pipeline on the benchmark. Iterate.

9. **Add: text-channel dedup, numeric detector, version detector.**
10. **Triage with cost budget.**
11. **Streamlit dashboard with calibration loop (full Stage 6).**
12. **Chroma adapter.**
13. **AWS deployment scripts.**

This order produces a working system at step 8, and every subsequent step makes it better without breaking what works.

11. Build instructions for Claude Code

Build pulse-scan v1 in the build order specified in §10. After each numbered step, stop and produce: (a) what was built, (b) how to run it locally against the fixture adapter, (c) any LLD assumptions that turned out to be wrong.

Do not proceed to the next step without my approval. Do not skip the synthetic 50-chunk benchmark in step 2 — it's the test corpus for everything after.

When the LLD leaves a choice flexible (specific Python libraries beyond those named, exact directory structure, CLI command surface, test framework), propose the choice with a one-line rationale and proceed unless I push back.

When the LLD specifies a choice firmly (the 7 stages and order, DuckDB + numpy memmap for control plane, three-detector contradiction approach, UMAP→HDBSCAN clustering, dashboard-driven calibration loop, Chroma as v1 target, Streamlit dashboard, GPU-required no-fallback, AWS architecture in §8), do not change it without explicit confirmation.